Sesión 20 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Aplicación funcional completa con entrada/salida

Nota

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

Parte A — Capa pura: el modelo de datos

Ejercicio 1. ★

Vamos a representar un gestor de tareas. Define los tipos de datos necesarios a partir de la siguiente descripción:

- Una **prioridad** puede ser baja, media o alta (constructores `Baja`, `Media` y `Alta`). Queremos poder mostrar prioridades en GHCi, compararlas con `==` y ordenarlas de forma que `Baja` sea la menor y `Alta` la mayor.
- Una **tarea** (tipo `Tarea`, con un único constructor llamado `CTarea`) está formada por una descripción (`String`), una prioridad y un booleano que indica si está completada, en ese orden. Queremos poder mostrar tareas en GHCi y compararlas con `==`.

A continuación, define mediante encaje de patrones las tres funciones de acceso a los componentes de una tarea, que usaremos en el resto de la hoja: `descripcion :: Tarea -> String`, `prioridad :: Tarea -> Prioridad` y `completada :: Tarea -> Bool`.

```
ghci> descripcion (CTarea "Estudiar Haskell" Alta False)
"Estudiar Haskell"
ghci> completada (CTarea "Estudiar Haskell" Alta False)
False
```

Por último, **antes** de evaluarlas en GHCi, predice el resultado de las siguientes expresiones y justifícalo. Después compruébalo.

```
ghci> compare Alta Media
ghci> maximum [Media, Baja, Alta, Baja]
ghci> CTarea "A" Alta False == CTarea "A" Alta True
```

Ejercicio 2. ★

Define `añadirTarea :: String -> Prioridad -> [Tarea] -> [Tarea]` que cree una tarea nueva (inicialmente no completada) y la añada al principio de la lista.

```
ghci> tareas = añadirTarea "Estudiar Haskell" Alta []
ghci> tareas
[CTarea "Estudiar Haskell" Alta False]
```

Ejercicio 3. ★

Define `completar :: String -> [Tarea] -> [Tarea]` que marque como completada la tarea cuya descripción coincida con la dada, dejando el resto sin modificar.

```
ghci> t2 = añadirTarea "Leer libro" Media tareas
ghci> completadas = completar "Estudiar Haskell" t2
ghci> map completada completadas
[False,True]
```

Ejercicio 4. ★

Define `pendientes :: [Tarea] -> [Tarea]` que devuelva solo las tareas no completadas, y `porPrioridad :: [Tarea] -> [Tarea]` que ordene las tareas de mayor a menor prioridad (por recursión, al estilo de `quicksort`).

```
ghci> length (pendientes completadas)
1
```

Parte B — Capa pura: consultas

Ejercicio 5.

Define `buscarTarea :: String -> [Tarea] -> Maybe Tarea` que devuelva la primera tarea cuya descripción coincida, o `Nothing` si no existe.

Ejercicio 6.

Define `contarPendientes :: [Tarea] -> Int` y `hayTareaUrgente :: [Tarea] -> Bool`, que comprueba si hay alguna tarea de prioridad `Alta` sin completar.

```
ghci> contarPendientes completadas
1
ghci> hayTareaUrgente completadas
False
```

Parte C — Capa IO: el programa interactivo

Ejercicio 7. ★

Define `mostrarTarea :: Tarea -> String` que formatee una tarea para mostrarla en pantalla, indicando su prioridad y si está completada.

```
ghci> mostrarTarea (CTarea "Ver shorts de YouTube" Media False)
"[ ] [MEDIA] Ver shorts de YouTube"
ghci> mostrarTarea (CTarea "Comprar PS5" Alta True)
"[x] [ALTA] Comprar PS5"
ghci> mostrarTarea (CTarea "Ordenar el escritorio" Baja False)
"[ ] [BAJA] Ordenar el escritorio"
```

Ejercicio 8. ★

Implementa el programa interactivo del gestor de tareas siguiendo el patrón visto en teoría: una función recursiva `bucle :: [Tarea] -> IO ()` que recibe como parámetro la lista de tareas actual (el *estado* del programa) y un `main :: IO ()` que arranca el bucle con la lista vacía. En

cada iteración, `bucle` debe mostrar el menú, leer la opción elegida con `getLine` y actuar según la opción:

1) **Añadir tarea.** Pide al usuario la descripción de la tarea y después su prioridad, que se introduce como una letra: `A` (alta), `M` (media) o `B` (baja). Si el usuario escribe cualquier otra cosa, se considera prioridad baja. Crea la tarea con `añadirTarea` y continúa el bucle con la lista resultante.
2) **Marcar como completada.** Pide la descripción de la tarea a completar y continúa el bucle con la lista que devuelve `completar`. Si ninguna tarea tiene esa descripción, la lista queda igual (no es necesario mostrar ningún mensaje de error).
3) **Mostrar pendientes por prioridad.** Muestra, una por línea, solo las tareas no completadas, ordenadas de mayor a menor prioridad (usa `pendientes` y `porPrioridad`). La lista de tareas no cambia.
4) **Mostrar todas.** Muestra, una por línea, todas las tareas (completadas o no) en el orden en que están en la lista. La lista de tareas no cambia.
5) **Solo tareas urgentes.** Muestra, una por línea, las tareas urgentes, es decir, las de prioridad `Alta` que todavía no están completadas. La lista de tareas no cambia.
6) **Salir.** Termina el programa sin volver a llamar a `bucle`.

Si el usuario introduce una opción distinta de las anteriores, muestra el mensaje `Opcion no valida` y vuelve a mostrar el menú sin modificar la lista. Para mostrar cada tarea en las opciones 3, 4 y 5 usa la función `mostrarTarea` del ejercicio anterior, y para mostrar varias líneas de golpe usa el patrón `putStr (unlines (map f xs))` visto en teoría.